

Modelo C4 de Arquitetura

SynapseEdu — Hub Inteligente de Recursos Educacionais

O modelo C4 (Context, Container, Component, Code) descreve a arquitetura do sistema em níveis crescentes de detalhe, cada um pensado para uma audiência diferente. Este documento cobre os três primeiros níveis — Code não é necessário para um projeto deste porte.

Nível 1 — Diagrama de Contexto (System Context)

Objetivo: mostrar o sistema como uma caixa-preta e como ele se relaciona com pessoas e sistemas externos.
Audiência: qualquer pessoa, técnica ou não.

Ator / Sistema	Tipo	Relação com o SynapseEdu
Conteudista	Pessoa (usuário)	Acessa o sistema via navegador para cadastrar, listar, editar e excluir recursos educacionais, e para usar o Smart Assist.
SynapseEdu	Sistema (o próprio)	Hub que centraliza o cadastro de materiais didáticos e usa IA para sugerir descrições e tags.
Google Gemini API	Sistema externo	Recebe título e tipo do material e retorna uma descrição sugerida e 3 tags, via chamada HTTP autenticada por API key.

Representação textual do diagrama:

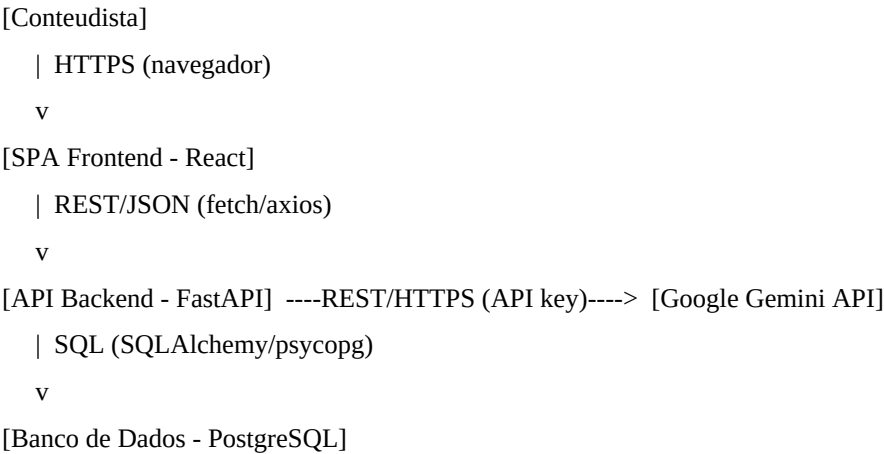
[Conteudista] --usa (navegador/HTTPS)--> [SynapseEdu] --consulta (REST/HTTPS)--> [Google Gemini API]

Nível 2 — Diagrama de Containers

Objetivo: "abrir" o sistema SynapseEdu e mostrar as principais peças executáveis/implantáveis (containers, no sentido C4 — não necessariamente Docker) e como elas se comunicam. Audiência: desenvolvedores e arquitetos.

Container	Tecnologia	Responsabilidade
SPA Frontend	React	Interface web onde o conteudista lista, cadastra, edita e exclui recursos, e aciona o Smart Assist. Consome a API REST do backend.
API Backend	Python 3 + FastAPI	Expõe endpoints REST (/api/v1/resources, /api/v1/resources/smart-assist, /health), valida payloads com Pydantic, aplica regras de negócio e orquestra a chamada à IA.
Banco de Dados	PostgreSQL	Persiste os recursos educacionais (tabela resources) via SQLAlchemy (ORM).
Serviço de IA (externo)	Google Gemini API	Gera descrição e tags a partir de um prompt de sistema ("Assistente Pedagógico"), respondendo em JSON estrito.

Representação textual do diagrama:



Nível 3 — Diagrama de Componentes (API Backend)

Objetivo: "abrir" o container API Backend e mostrar seus principais módulos internos e como colaboram.
Audiência: desenvolvedores que vão mexer diretamente no código do backend.

Componente	Módulo	Responsabilidade
Routers	app/routers/resources.py	Define os endpoints HTTP, recebe/valida requisições (via Schemas) e delega a lógica para CRUD ou Services. Não conhece detalhes de SQL nem da API de IA.
Schemas	app/schemas/resource.py	Modelos Pydantic que definem o contrato de entrada/saída da API (validação de tipos, formatos de URL, etc.).
CRUD	app/crud/resource.py	Funções que executam operações de leitura/escrita no banco via SQLAlchemy. Não conhece detalhes de HTTP.
Models	app/db/models.py	Definição das tabelas do banco (ORM) — mapeamento objeto-relacional da entidade resources.
AI Service	app/services/ai_service.py	Monta o prompt de sistema ("Assistente Pedagógico"), chama a API do Gemini, faz o parsing do JSON de resposta e trata falhas/timeouts.
Config	app/core/config.py	Centraliza a leitura de variáveis de ambiente (.env): DATABASE_URL, GEMINI_API_KEY, etc.
Database	app/db/database.py	Cria a engine e as sessions do SQLAlchemy usadas pelo CRUD para conversar com o PostgreSQL.

Representação textual do fluxo (exemplo: requisição de Smart Assist):

[Router: POST /smart-assist] --> [Schema valida entrada] --> [AI Service monta prompt e chama Groq] --> [AI Service faz parsing do JSON] --> [Router retorna description + tags ao Frontend]

Representação textual do fluxo (exemplo: criação de recurso):

[Router: POST /resources] --> [Schema valida entrada] --> [CRUD insere via Models] --> [Database (session)] --> [PostgreSQL]

Nível 2b — Vista de Implantação (Deployment)

Complementa o Nível 2 mostrando onde cada container roda fisicamente em produção — útil para quem for reproduzir o deploy ou depurar um problema de rede entre serviços.

Container	Plataforma	Observações
SPA Frontend	Vercel	Build estático gerado por npm run build, servido via CDN da Vercel. Variável VITE_API_BASE_URL aponta para a URL pública do backend no Render.
API Backend	Render (Web Service, Docker)	Construído a partir de backend/Dockerfile. Porta injetada dinamicamente via variável PORT do Render. CORS restrito explicitamente ao domínio da Vercel.
Banco de Dados	Neon (PostgreSQL gerenciado)	Mesma instância usada em desenvolvimento local sem Docker (ver ADR-05). Conexão via DATABASE_URL com sslmode=require.
Serviço de IA	Groq API (externo)	Consumido diretamente pelo backend no Render; não requer infraestrutura própria (ver ADR-06).

Representação textual do diagrama de implantação:

[Navegador do usuário] --HTTPS--> [Vercel: SPA Frontend]
[Vercel: SPA Frontend] --HTTPS/REST (CORS liberado)--> [Render: API Backend]
[Render: API Backend] --SQL/SSL--> [Neon: PostgreSQL]
[Render: API Backend] --HTTPS--> [Groq API]

Nota: o docker-compose.yml usado em desenvolvimento local utiliza um PostgreSQL em container isolado (não o Neon), por decisão de segurança registrada no ADR-05. O ambiente de produção (Render) é o único configurado para usar o Neon diretamente.

Por que parar no Nível 3?

O C4 possui um quarto nível (Code), que detalharia classes e métodos individuais — geralmente gerado automaticamente por ferramentas de IDE (ex.: diagramas UML) e não mantido manualmente. Para o porte deste projeto, os níveis Contexto, Container e Componente já fornecem clareza suficiente para qualquer desenvolvedor entender o sistema sem precisar ler o código-fonte.